

AI AGENT 项目实战营 · 第 07 节

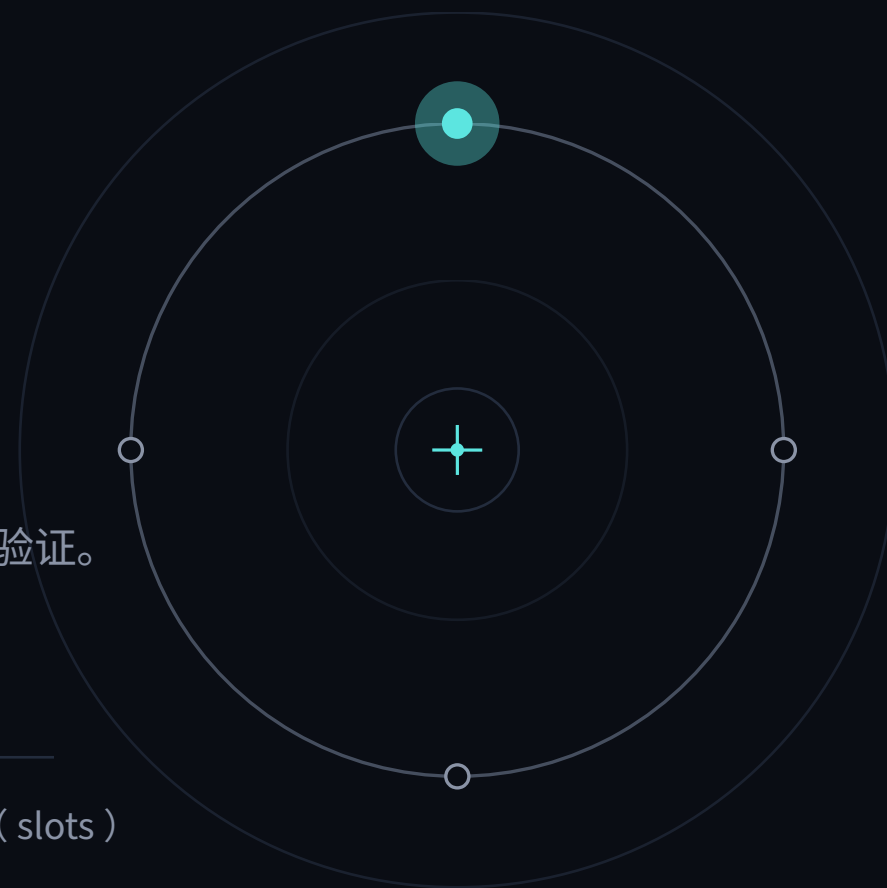
Session 与短期记忆

让 Agent 记得住你刚说过的话

把完整历史存好，同时控制本轮真正给模型看的输入；最后用 SDK usage 验证。

消息历史 · SQLiteSession · 窗口预算 · tiktoken · 滑窗 / 摘要 · 结构化 state (slots)

LECTURE · UNIT 3 · 60 MIN



它失忆了

第 1 轮

你 你好，我叫小明。

助手 你好小明，很高兴认识你！

第 2 轮

你 我刚才说我叫什么名字？

助手 抱歉，我不知道你的名字，方便再告诉我一次吗？

← 上一秒还在叫『小明』，这一秒就忘了。

问题 它不是不在乎——它是结构上『记不住』。这节课，我们就来修这个。

没记忆没人用



体验崩塌

每轮都要重新自我介绍，像跟金鱼聊天



工具也白搭

上节加了工具，但它记不住你上一步要干嘛



岗位硬要求

『上下文 / 记忆工程』是 Agent JD 高频词

从『单轮玩具』到『能连续对话的助手』，差的就是这块——短期记忆。

今天七步

01



失忆从哪来

LLM 为什么无状态

02



消息历史

用 messages 拼出记忆

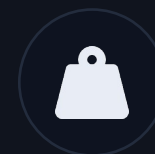
03



Sessions

SQLiteSession 自动存取

04



窗口预算

token 有限, 会爆

05



裁剪策略

滑窗 · 摘要 · state 档案

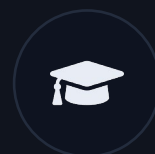
06



上手

6 个主线 + 3 个工程 demo

07



作业分层

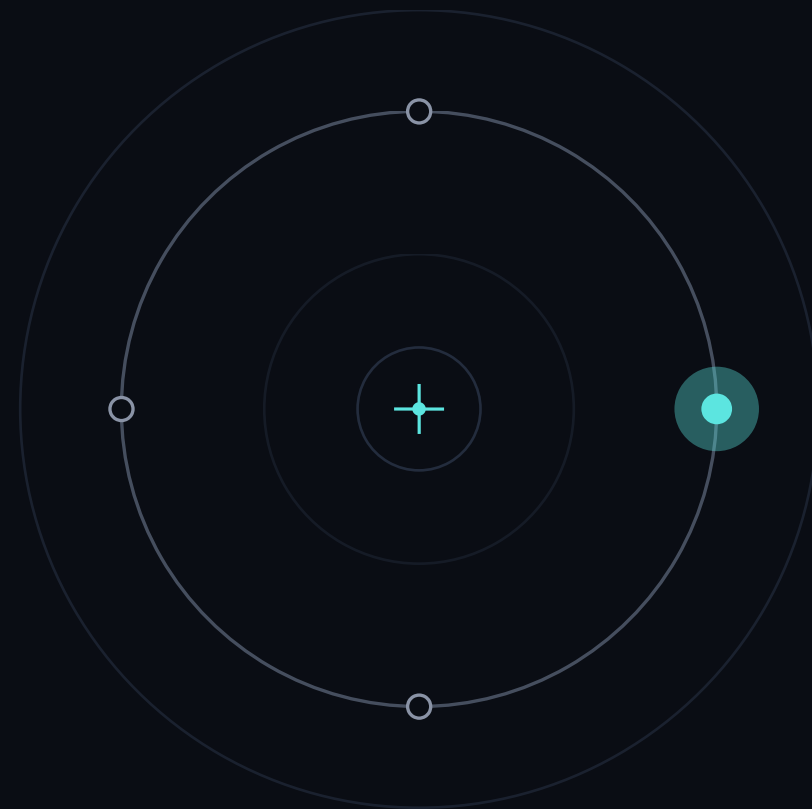
基础 Goal + 进阶挑战

01

PART 1 · THE PROBLEM

失忆从哪来

每次 run，都是一次全新的开始



LLM 是无状态的

一句话 模型不会『记得』上一次——每次调用，它只看你这一次喂进去的东西。

run #1
『我叫小明』

×

run #2
『你叫什么?』

两次 run 之间，没有任何东西被自动带过去。

那 ChatGPT 怎么记得住？它在你看不见的地方，把整段历史每轮重新拼进去喂给模型。

所谓『短期记忆』，本质就是这件事：自己维护一份对话历史，每轮带上。

每轮重喂历史

第 1 轮 [system] + [user: 我叫小明]

第 2 轮 [system] + [user: 我叫小明] + [assistant: 你好] + [user: 我叫什么]

第 3 轮 ...前面全部 + 这一轮的新消息

注意 模型每轮都是『从零读一遍全部历史』——这既是它能记住的原因，也是历史越长越贵的根源。

短期记忆 vs 长期记忆

维度	短期记忆（本节）	长期记忆（第 8-9 节）
范围	就这一次会话内	跨会话，下次还记得
存哪	对话历史（messages 列表）	外部存储 / 向量数据库
怎么取	每轮整段带上	按相关性检索召回
典型内容	刚说的话、上一步结果	你的长期偏好、技能栈、底线

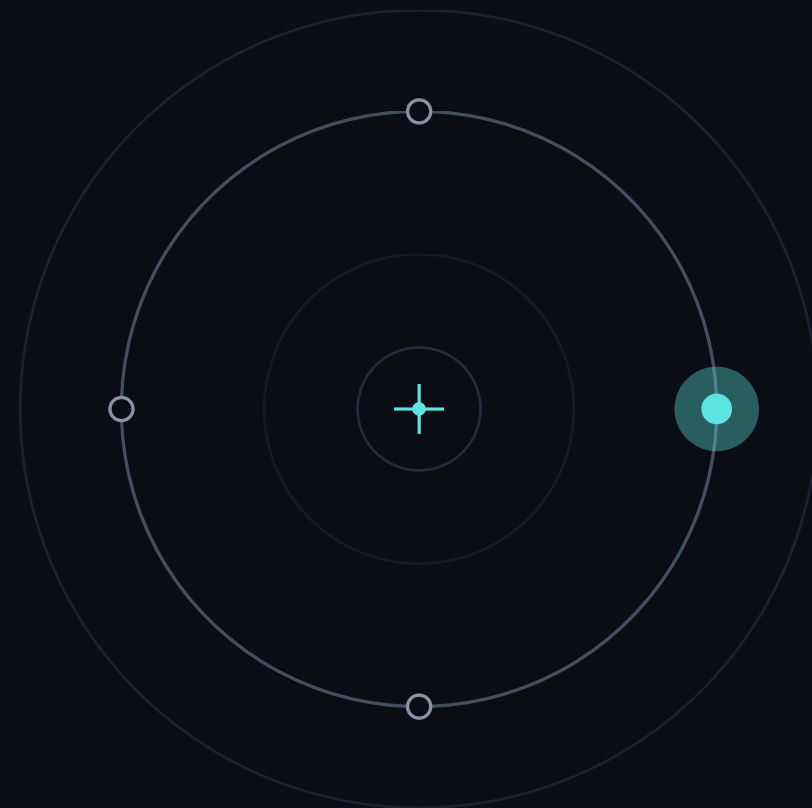
今天只做『短期』；下节起做『长期』。两者合起来 = P3 双层记忆。

02

PART 2 · MESSAGE HISTORY

消息历史 = 短期记忆

记忆不是魔法，是一个列表



一次请求怎么拼

system	系统提示 你是谁、能做什么（角色卡，第 3 节）	角色
user	用户消息 用户这一轮说的话	输入
assistant	助手消息 模型上一轮的回答	历史
tool	工具返回 上节的工具跑完，结果也是一条消息进历史	结果

这串列表每轮都是重新拼出来的—— user / assistant / tool 不断往后追加，越聊越长。

打印记忆看一眼

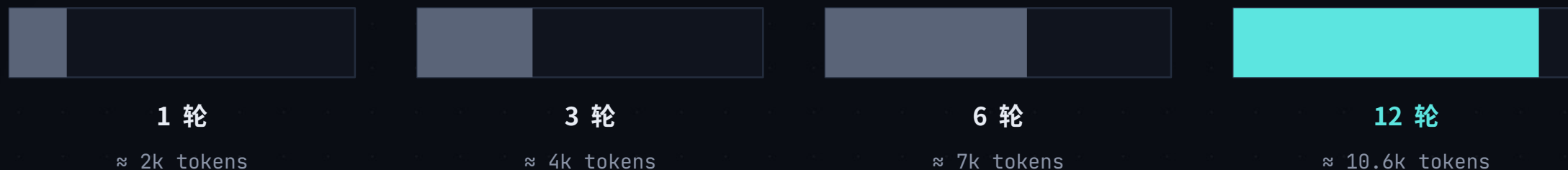
```
# 短期记忆 = 一个 messages 列表
items = [
    {role: "user",
     content: "我叫小明"},
    {role: "assistant",
     content: "你好小明"},
    {role: "user",
     content: "我叫什么? "},
]
```

要点

- 模型每轮真正『看到』的，就是这串列表
- 要『记得住』，就把它每轮重新喂进去
- 多聊一轮，列表就 +2 条（user+assistant）
- 所以它会一直变长 —— 埋下隐患（Part 4）

配套 demo 03_inspect_history.py · 离线可跑

每轮 +2 条



线性增长 历史是累加的：聊得越久，每一轮要喂给模型的东西越多—— token、延迟、费用全都跟着涨。

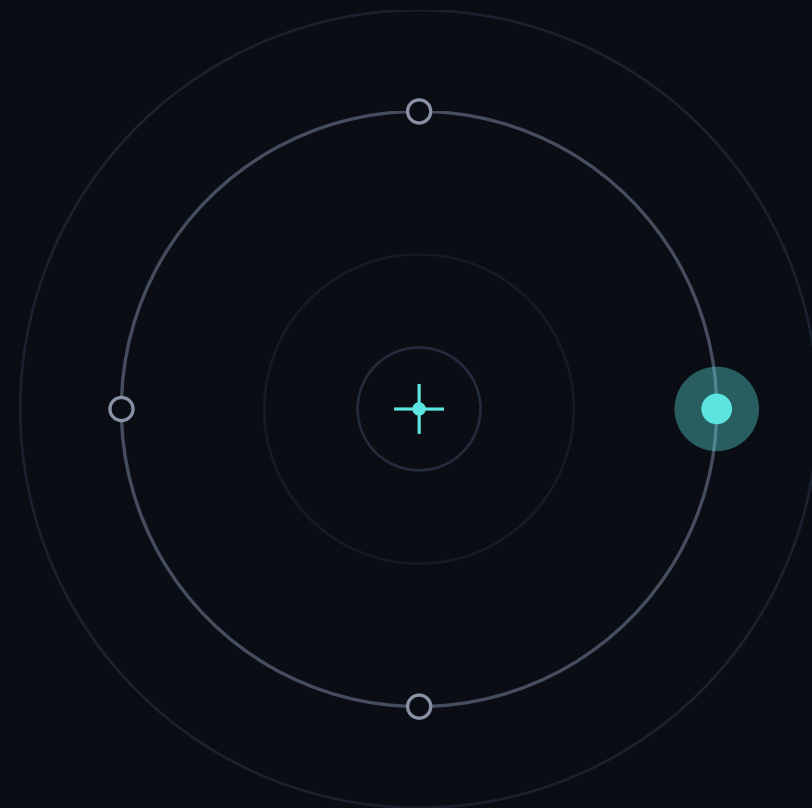
这就是为什么必须有人决定『留什么、扔什么』—— Part 5 的裁剪。

03

PART 3 · SDK SESSIONS

Sessions：自动帮你记

不用手搓历史，一行就够



SQLiteSession 做了什么

SQLiteSession · 自动存取

你只管 run，它替你做三件事：

1 run 之前

自动把历史拼回 prompt

2 run 之后

把这轮 user+assistant 存起来

3 存哪

SQLite（内存或落盘文件）

它的方法 · 想自己操作历史时用

`get_items()` 取回全部历史（list）

`add_items([...])` 手动追加消息

`pop_item()` 弹出最后一条

`clear_session()` 清空这个会话

滑窗 / 摘要就是基于 `get_items()` 做裁剪（Part 5）。

加记忆，只多一行

```
from agents import Agent, Runner, SQLiteSession

agent = Agent(name="ResuMatch", instructions=...)
session = SQLiteSession("alice:thread-01")

# 同一个 session 贯穿多轮 —— 关键就是 session=
Runner.run_sync(agent, "我只看远程岗位", session=session)
r = Runner.run_sync(agent, "我刚说的地点要求?",
session=session)
print(r.final_output)    # → 只看远程岗位
```

DIFF

对比 01_no_memory.py ,

唯一的区别就是

session=session

这一个参数。

想跨进程留存就落盘：

```
SQLiteSession("id",
               db_path="mem.db")
```

→ demo 02_sqlite_session.py

隔离边界是 tenant/user/thread

坑 多个用户复用同一个 session_id，张三和李四的记忆就会混在一起——张三问问题，模型拿李四的历史回答。

```
# 同一用户也可能同时开多个会话
sid = f"{tenant_id}:{user_id}:{thread_id}"
session = SQLiteSession(sid, db_path="mem.db")
```

规则： session_id 用来定位会话，不是权限系统。读写前仍要检查当前用户是否有权访问这个 thread。

隔离做错不是『答偏了』，而是跨用户数据泄漏。

Session 放哪，取决于怎么跑

运行方式	先用什么	为什么
单机作业 / 本地 Demo	SQLite	零运维，能落盘，最容易看懂
多进程 Web 服务	Redis / 数据库	多个进程能读到同一份会话
已有业务数据库	SQLAlchemySession	沿用连接、备份和权限体系
需要服务端续接	Conversations / response id	少搬历史，但要明确状态归谁管理

判断顺序：先看是否跨进程，再看是否要持久化，最后才看性能。SQLite 不是『低级版』，而是单机场景的正确答案。

会话不能只会新增

ACTIVE	正在聊	正常读写，更新 last_seen
IDLE	一段时间没动	可在空闲时压缩，减少下次读取
ARCHIVED	明确结束	只读保留，默认不再拼进模型
DELETED	用户删除 / 到期	真正清除历史和派生缓存

TTL（保留期）是业务规则，不是数据库默认值。课程项目至少写清：多久算过期、谁能删、删完如何验证。

两个标签页会把顺序写乱

没有版本检查

- A 标签读到 version=7，开始生成
- B 标签也读到 version=7，先写成 8
- A 晚回来，又按旧状态写入
- 历史顺序和用户看到的顺序不一致

这叫竞态，不是模型幻觉。

用版本号做乐观并发控制

```
UPDATE sessions  
SET version = version + 1  
WHERE id = ? AND version = ?
```

更新行数 = 0，说明有人抢先写了。重新读、重放或提示用户，不要悄悄覆盖。

一次请求，最多写入一次

真实情况 网络超时后，前端会重试。同一个问题可能到服务器两次，工具副作用也可能执行两次。

```
request_id = client_generated_uuid
UNIQUE(session_id, request_id)
if already_done: return saved_result
else: run_once_and_commit()
```

幂等保护什么

- ✓ 重复消息不进历史
- ✓ 邮件 / 支付 / 写库不执行两遍
- ✓ 重试返回第一次已经保存的结果

demo 07 用 SQLite 的 UNIQUE 约束演示：第二次提交同一个 request_id，记录数仍然是 1。

改问题，不要把错误历史留着

需求	做法	适合什么情况
撤回最后一问	pop_item() 两次	去掉 assistant 回答和对应 user 问题
从旧节点重试	复制到新 thread / branch	保留原线，比较两条后续
只改当前档案	更新 state，不改历史	事实纠正，但仍要保留发生过什么
彻底删除敏感内容	删除源记录和派生副本	隐私请求，不能只在界面上隐藏

『历史』是事件记录，『state』是当前事实。修正前先问：我要改发生过什么，还是改现在是什么？

压缩最好放在用户等不到的地方

每轮结束立刻压缩

- 实现简单，历史总能保持短
- 但压缩本身也要一次模型调用
- 流式答案结束后，连接可能还在等
- 高频聊天会把压缩成本放大

空闲时 / 达到阈值再压缩

- ✓ 先把本轮答案尽快交给用户
- ✓ 按 token、轮数或空闲时间触发
- ✓ 失败可重试，不影响主回答
- ✓ 压缩前后记录版本，方便回查

官方 compaction session 也提醒：自动压缩会让 streaming 在最后一个 token 后继续等待。

会话落盘后，就按用户数据对待

最小化

只存继续任务需要的内容；大文件存引用，不把全文塞历史

加密

本地文件也可能被拷走；需要用 EncryptedSession 包一层

保留期

到期后不再返回只是第一步，还要安排物理清理

删除验证

删 session、摘要、缓存和索引后，再用同一查询确认找不到

加密解决『被拿走能不能看』；权限解决『谁能拿走』。两件事不能互相替代。

记忆功能至少测这六件事

01 重启

进程重启后，历史还在

02 隔离

Alice 看不到 Bob 的任何消息

03 并发

两个标签页同时写，不静默覆盖

04 重试

相同 request_id 只落一次

05 长聊

超过预算后仍能回答，硬约束不丢

06 删除

清理后搜索、摘要和缓存都找不到

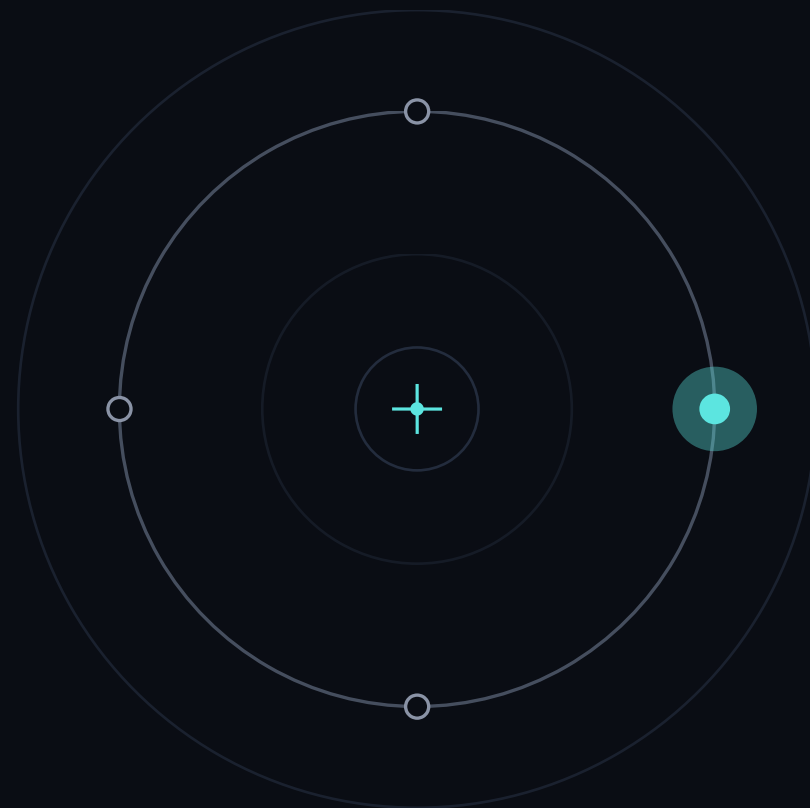
能连续对话只是起点；可恢复、可隔离、可重试、可删除，才算一个完整的 Session 功能。

04

PART 4 · TOKEN BUDGET

上下文窗口是有限的

记得住，也不能无限地记



token 是什么，怎么数

token $\approx$ 文本的『块』

- 模型不按『字』算，按 token 算
- 英文：1 token $\approx$ 4 个字符 / 0.75 词
- 中文：1 个汉字 $\approx$ 1 ~ 2 token
- 标点、空格也各占 token

『一条消息多长』 $\neq$ 『几个字』，得真去数 token。

用 tiktoken 做本地估算

```
import tiktoken
enc = tiktoken.encoding_for_model(
    MODEL)
n = len(enc.encode(text))
# 本地预算估算；最终以 API usage 为准
```

不要把某个固定 tokenizer 的结果称为『绝对真实』；模型与消息封装都会影响最终计费。

按条数裁不可控

✗ 按消息条数裁

『只留最近 6 条消息』听起来可控，其实不是：

- 6 条闲聊 $\approx$ 几百 token
- 6 条里夹一段长 JD $\approx$ 几千 token
- 同样『6 条』，token 能差 10 倍
- $\rightarrow$ 你永远不知道会不会爆窗口

条数稳定 $\neq$ token 稳定。

✓ 按 token 预算裁

给历史定一个 token 预算（如 3000），裁到预算内：

- 真实反映『会不会爆、会花多少钱』
- 长消息自动被多裁、短消息少裁
- 这才是工程意义上的『可控』

下一页：trim_to_token_budget。

先估算，再看真实 usage

```
import tiktoken
enc = tiktoken.encoding_for_model(MODEL)

def trim_to_token_budget(items, budget):
    kept, total = [], 0
    for m in reversed(items): # 从新到旧
        t = len(enc.encode(m["content"]))
        if total + t > budget: break
        kept.append(m); total += t
    return list(reversed(kept))
```

思路

从最新消息往回收，

累加 token，到预算就停。

长消息吃预算快 → 自动少留几条；

短消息 → 自动多留。

本地计数用于预算；最终以
`result.context_wrapper.usage` 为准。

窗口预算怎么分

system

角色卡

固定开销，不裁

历史

对话记忆

主要裁剪对象

新问题

用户这轮

必留

回答留白

给模型输出

必须预留，不然没空间答

裁剪几乎只动『历史』这一块——其它三块要么固定、要么必留。

不要估账，直接读 usage

✗ 不裁：历史一路累加

- 每次 Runner.run 都有独立 usage
- input_tokens 包含本轮重发的历史
- 工具调用和 handoff 也会计入
- 连续跑 20 轮，观察曲线是否持续上升

→ 先拿数据，再判断是不是历史导致成本上涨。

✓ 把 usage 打出来

- usage.input_tokens：本轮输入
- usage.output_tokens：本轮输出
- request_usage_entries：逐次模型调用
- 裁剪前后用同一段脚本对照

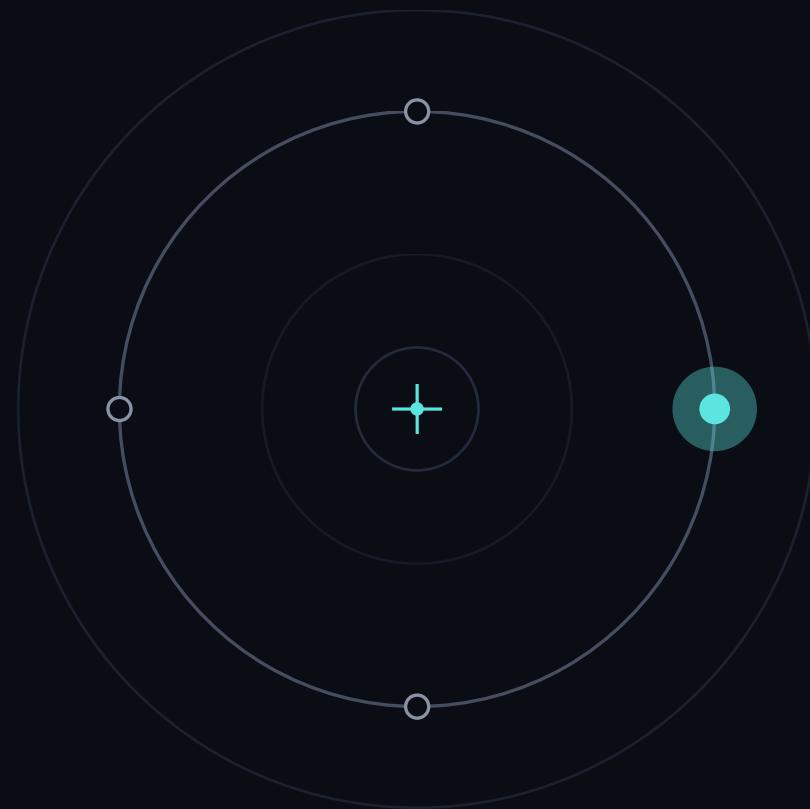
→ 没有 usage 证据，就不要写『节省 70%』。

05

PART 5 · TRIMMING

两招裁剪历史

留住有用的，丢掉冗余的



只让模型看最近历史

轮 1

轮 2

轮 3

轮 4

轮 5

轮 6

丢弃

保留最近 3 轮（窗口）

怎么做

`RunConfig(session_input_callback=...)`

完整历史继续保存在 Session；callback 只决定本轮给模型看什么。

✓ **优点** 实现最简单、好理解

× **代价** 更早的信息直接丢（如开头说的名字）；条数固定但 token 仍不定

DEMO

```
def keep_recent(history, new_input):
```

```
    return history[-6:] + new_input
```

```
RunConfig(session_input_callback=keep_recent)
```

04_sliding_window.py · 完全离线可跑

会话太长，先压成一份简版历史

完整 Session（继续保存）



平台压缩（compact）



压缩结果 + 最近对话

怎么做

直觉：把前面很多轮折成一份更短、仍能继续任务的状态。代码上用 `OpenAIResponsesCompactionSession`；高级用法再考虑手动触发。

✓ **优点** 和 Responses 模型的长任务状态保持一致

× **代价** 压缩会增加一次处理；自动压缩可能拖长 streaming 收尾

RESULT

```
underlying = SQLiteSession(...)
```

```
session = OpenAIResponsesCompactionSession(...)
```

```
await session.run_compaction({"force": True})
```

05_summary_memory.py · 默认看数据流；--live 才真压缩

摘要别丢硬约束

风险 摘要是『有损压缩』。压太狠，会把决定成败的硬约束丢掉——而它看起来只是『一句小细节』。

哪些是『硬约束』

- 健身：膝盖不好 → 不能深蹲
- 求职：只看国内岗位 / 远程
- 投研：不接受高风险 / 只看 A 股
- 通用：预算上限、时间死线、过敏源

做法：摘要 prompt 点名要保留

"" 把以下对话压成 3 句，
但必须原样保留：
用户的硬性限制、数字、
不可逾越的底线。 ""

把『必须保留什么』写死进摘要指令——别指望模型自己判断什么重要。

三个控制点，各管一层

控制点	解决什么	默认选择
SessionSettings(limit)	先取多少历史	条数上限 / 保护数据库读取
session_input_callback	本轮给模型看什么	裁剪、重排、按 token 预算
CompactionSession	长任务如何跨窗口	Responses 路线优先原生能力
手写摘要	自定义业务保留规则	确实需要可读、可审计时再写

不要把『存储多少』和『模型本轮看多少』绑成一个开关。

裁剪挂在哪一步

```
def keep_recent(history, new_input):  
    return history[-6:] + new_input  
  
run_config = RunConfig(  
    session_input_callback=keep_recent,  
    session_settings=SessionSettings(limit=50),  
)  
await Runner.run(agent, q, session=s, run_config=run_config)
```

关键认知

Runner 先从 Session 取历史，

再调用 callback 合并新输入。

callback 返回的列表，

就是模型本轮真正看到的 items。

这才算把裁剪接进请求路径。

同一段脚本，比较 usage

Session 里完整保存



12 条历史 items · demo 04 实测

- 六轮对话完整落在 SQLite
- 便于恢复、审计和重新处理
- 保存数量不等于本轮输入数量

callback 返回的模型输入



7 条 input items · 最近 6 条 + 新问题

- Session 仍然保留全部 12 条
- 模型本轮只看 callback 的返回值
- token 差异再读取 API usage

先验证请求路径确实变了，再比较 input_tokens；不要把 item 数直接当 token 数。

原生压缩还是手写摘要



默认路线 Responses API + 原生 compaction ; 模型升级时压缩机制也一起升级。

需要人可读摘要、跨模型兼容或明确业务字段时，才维护自己的 summarizer 。跨会话事实仍交给 L8-L9 。

状态和历史分开存

思路 历史记录发生过什么；state 记录当前是什么。地点、签证和薪资底线不要赌摘要是否保得住。

消息历史 · 滑窗后只剩最近 2 条

user: 我叫小明，目标减脂 5 公斤

user: 我膝盖不好，别安排深蹲

user: 周三具体练什么？

assistant: 胸背为主，轻重量多组数

名字和底线，跟着被裁的旧消息一起没了。

结构化 state · 硬事实档案

name 小明

goal 减脂 5 公斤

hard_line 膝盖不好，避免深蹲

工具写进来 · 不占历史 · 默认不给模型看，注入才看见

每轮拼 prompt 时把档案附上：历史随便裁，名字和底线永不丢。

state：写进去，每轮带上

```
@function_tool
def save_profile(w: RunContextWrapper[Profile],
                field: str, value: str) → str:
    setattr(w.context, field, value) # tool → slots
    return "saved"

def with_profile(ctx, agent): # rebuilt EVERY turn
    return BASE + f"\nProfile: {vars(ctx.context)}"

coach = Agent[Profile](instructions=with_profile, ...)
```

为什么值得

裁剪安全网

硬约束不再赌摘要写得好不好

敏感可控

大结果留在 state，注入时挑着给

省 token

几十 token 的档案，替掉反复重发的长返回

06_state_slots.py · 历史清零也记得你

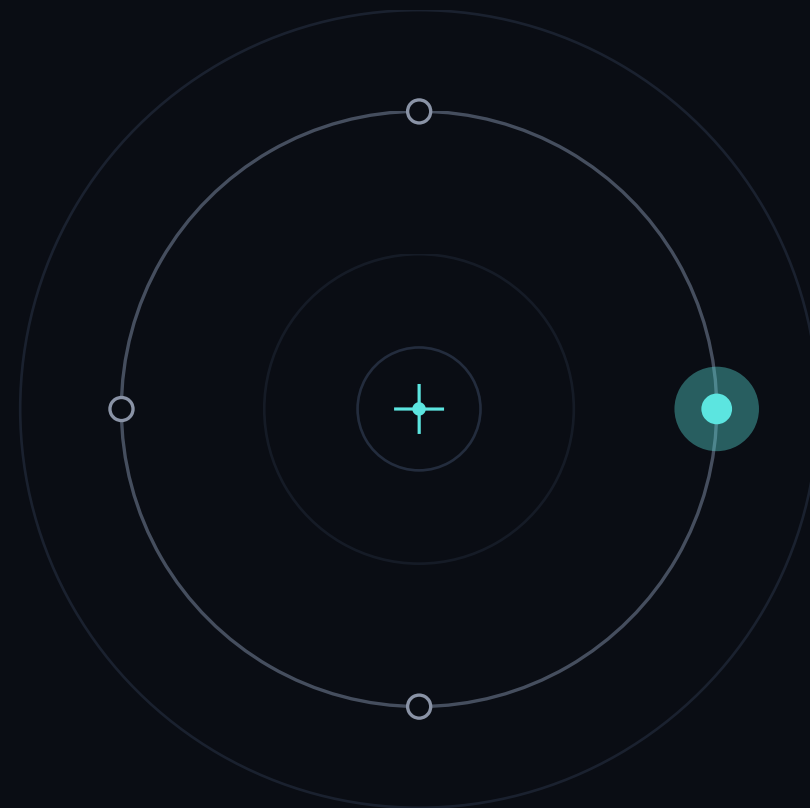
SDK 原语：context 对象（模型默认看不到）+ 动态 instructions（每轮现拼、把 state 带上）。

06

PART 6 · HANDS-ON

上手：把 Session 做完整

先跑 6 个主线，再用 3 个工程 demo 补可靠性



九个演示文件

01 no_memory.py

失忆现场：第 2 轮就忘了你是谁



02 sqlite_session.py

加 SQLiteSession，一行就记得住



03 inspect_history.py

打印 session 里存了哪些消息



04 sliding_window.py

自己实现『只留最近 N 轮 / token』



05 summary_memory.py

摘要压缩 · 你在自己 Agent 上加滑窗



06 state_slots.py

★ 进阶 · 结构化 state：历史清零也记得你

主线 01-06 先学记忆与裁剪；工程补充 07-09 全离线，分别演示幂等、并发版本和生命周期。

三个离线 demo：把边界情况跑出来

07

`session_idempotency.py`

同一 request_id 提交两次，数据库只留一条

08

`session_concurrency.py`

两个写入都拿旧 version，第二个明确冲突

09

`session_lifecycle.py`

active → archived → deleted，状态变化可验证

这三份代码不调用模型：先把存储层的行为测对，再接 Agent。这样出错时，你知道该查数据库还是查 prompt。

demo 走查

01 no_memory

- 先复现失忆：第 2 轮就忘了名字
- 确认问题是结构性的，不是模型笨
- 对照组，让 02 的改进看得见

先跑这个『坏例子』，才知道后面修好了什么。

02 sqlite_session

- 只加 session=session 一个参数
- 第 2 轮立刻记得住名字了
- 对比 01：改动小、效果大

一行代码，从『金鱼』变『记得住』。

动手：加滑窗

01



跑 01 → 04

看懂『失忆 → session → 看历史 → 滑窗』

02



改 05 加载剪

把滑窗 / 摘要套到你选题的 Agent

03



跑一段长对话

验证 token 不暴涨、关键信息没丢

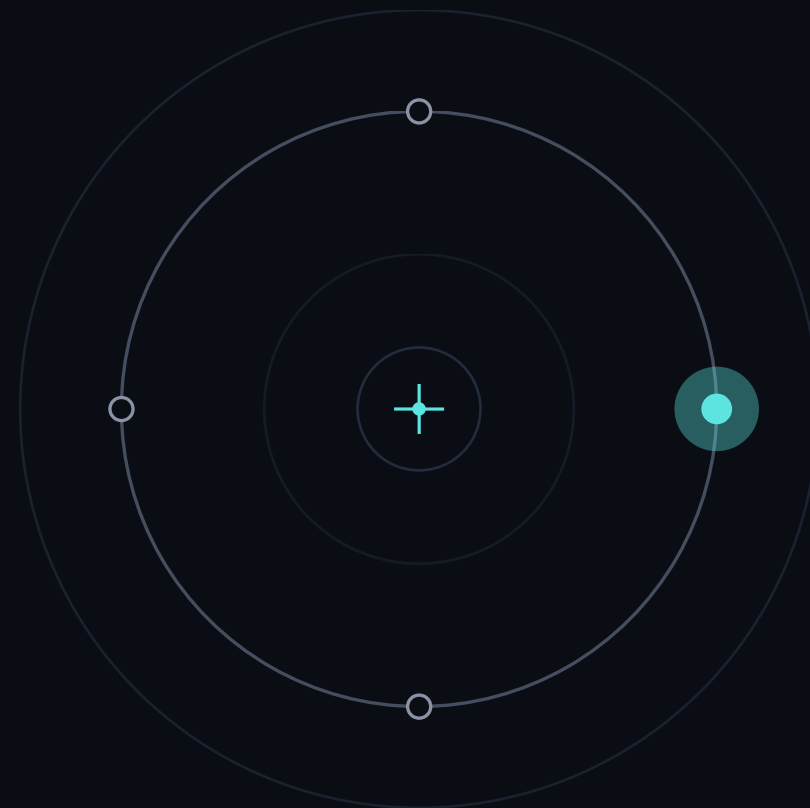
05 文件底部有标了 ★ TODO 的填空处，照着提示把裁剪接到你的 Agent。

07

PART 7 · ASSIGNMENT

作业分层

基础必达 + 进阶挑战，按你的时间挑



基础必做

🎯 **本节基础目标** 给你选题的 Agent 加上 SQLiteSession 短期记忆，并实现一种裁剪（滑窗），长对话不失忆、不暴涨。



跑通 demo 01-04

截图，理解失忆→session→历史→滑窗



给 Agent 加 session

你选题的 Agent 能记住多轮



加一种裁剪

滑窗（进阶可按 token）

时间紧（如 2h/ 周）：做到『加 session + 滑窗』就是合格，token 裁剪留作进阶。

进阶加分



打印 token 数

用 tiktoken 实时显示当前历史多少 token



按 token 预算裁

实现 trim_to_token_budget，遇长消息不爆



state 档案卡

照 demo 06：硬约束写进 state 每轮注入——历史怎么裁都不丢



滑窗 + 摘要 + state

三招组合跑 20 轮：token 稳、底线不丢（摘要 prompt 点名保硬约束）

时间足的压满组合挑战；时间紧、能力强的挑一项做深（state 档案卡 或 token 裁剪）。

三个最常踩的坑



无限堆历史

不裁剪 → token 暴涨 → 变慢、变贵、最后报错。上滑窗或按 token 裁。



摘要丢关键

压得太狠会把『只看国内岗位』这类约束丢了。摘要 prompt 里点名要保留的事实。



会话串味

多个用户复用同一个 session_id → 张三李四记忆混在一起。每个会话独立 id。

下节预告



今天你拿下了

- ✓ 失忆的根因：LLM 无状态
- ✓ 消息历史 = 短期记忆（一个列表）
- ✓ SQLiteSession 一行加记忆 + 会话隔离
- ✓ token 预算 + tiktoken + 滑窗 / 摘要裁剪
- ✓ 结构化 state：硬事实和历史分开存

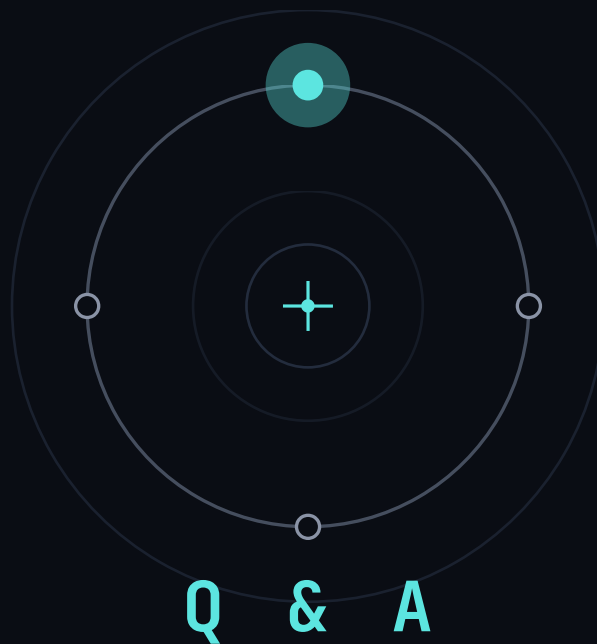


下节课 · 第 8 节

长期记忆与向量数据库 —— Embedding 与 ChromaDB 检索

- ✓ 短期记忆只在本次会话；长期记忆跨会话记住你
- ✓ 用向量检索把『相关的旧记忆』召回来

课前必看 · 《Demo 07 · ChromaDB 与 Embedding 实战》 25'



从『每轮失忆』到『记得住、还懂按 token 省着记』。

跑不通的，把报错贴群里 —— 我们一个个解决。

作业：跑通 01-04 · 给 Agent 加 session+ 滑窗 · 再从 06-09 任选一个工程边界做测试